

准的。成员 `d_name` 是文件名，`d_ino` 是文件位置。

如果出错，则 `readdir` 返回 `NULL`，并设置 `errno`。可惜的是，唯一能区分错误和流结束情况的方法是检查自调用 `readdir` 以来 `errno` 是否被修改过。

```
#include <dirent.h>

int closedir(DIR *dirp);
```

返回：成功为 0；错误为 -1。

函数 `closedir` 关闭流并释放其所有的资源。图 10-11 展示了怎样用 `readdir` 来读取目录的内容。

```
code/io/readdir.c

1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      DIR *stream;
6      struct dirent *dep;
7
8      stream = Opendir(argv[1]);
9
10     errno = 0;
11     while ((dep = readdir(stream)) != NULL) {
12         printf("Found file: %s\n", dep->d_name);
13     }
14     if (errno != 0)
15         unix_error("readdir error");
16
17     Closedir(stream);
18     exit(0);
19 }
```

code/io/readdir.c

图 10-11 读取目录的内容

## 10.8 共享文件

可以用许多不同的方式来共享 Linux 文件。除非你很清楚内核是如何表示打开的文件，否则文件共享的概念相当难懂。内核用三个相关的数据结构来表示打开的文件：

- 描述符表(descriptor table)。每个进程都有它独立的描述符表，它的表项是由进程打开的文件描述符来索引的。每个打开的描述符表项指向文件表中的一个表项。
- 文件表(file table)。打开文件的集合是由一张文件表来表示的，所有的进程共享这张表。每个文件表的表项组成(针对我们的目的)包括当前的文件位置、引用计数(reference count)(即当前指向该表项的描述符表项数)，以及一个指向 v-node 表中对应表项的指针。关闭一个描述符会减少相应的文件表表项中的引用计数。内核不会删除这个文件表表项，直到它的引用计数为零。

- v-node 表(v-node table)。同文件表一样，所有的进程共享这张 v-node 表。每个表项包含 stat 结构中的大多数信息，包括 st\_mode 和 st\_size 成员。

图 10-12 展示了一个示例，其中描述符 1 和 4 通过不同的打开文件表表项来引用两个不同的文件。这是一种典型的情况，没有共享文件，并且每个描述符对应一个不同的文件。

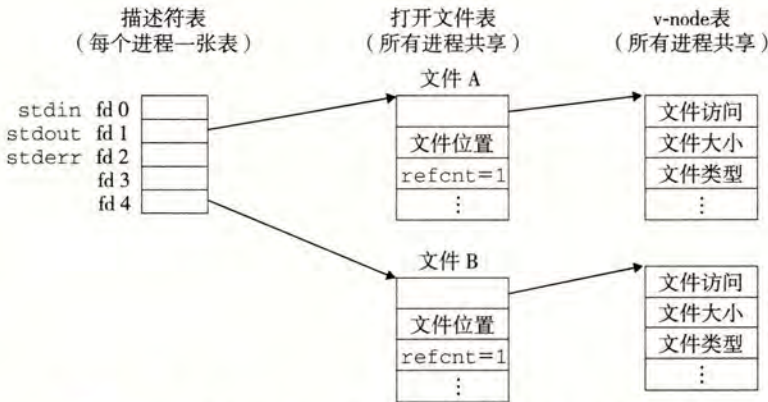


图 10-12 典型的打开文件的内核数据结构。在这个示例中，两个描述符引用不同的文件。没有共享

如图 10-13 所示，多个描述符也可以通过不同的文件表表项来引用同一个文件。例如，如果以同一个 filename 调用 open 函数两次，就会发生这种情况。关键思想是每个描述符都有它自己的文件位置，所以对不同描述符的读操作可以从文件的不同位置获取数据。

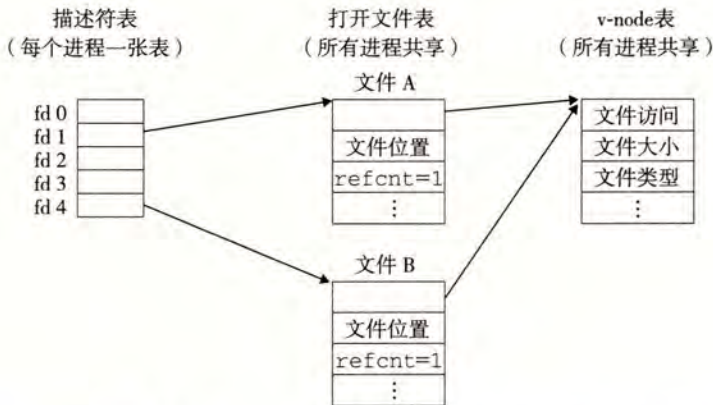


图 10-13 文件共享。这个例子展示了两个描述符通过两个打开文件表表项共享同一个磁盘文件

我们也能理解父子进程是如何共享文件的。假设在调用 fork 之前，父进程有如图 10-12 所示的打开文件。然后，图 10-14 展示了调用 fork 后的情况。子进程有一个父进程描述符表的副本。父子进程共享相同的打开文件表集合，因此共享相同的文件位置。一个很重要的结果就是，在内核删除相应文件表表项之前，父子进程必须都关闭了它们的描述符。



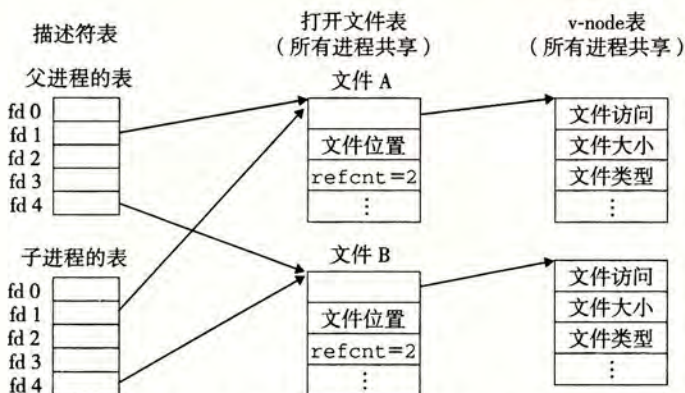


图 10-14 子进程如何继承父进程的打开文件。初始状态如图 10-12 所示

 **练习题 10.2** 假设磁盘文件 `foobar.txt` 由 6 个 ASCII 码字符“foobar”组成。那么，下列程序的输出是什么？

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6      char c;
7
8      fd1 = Open("foobar.txt", O_RDONLY, 0);
9      fd2 = Open("foobar.txt", O_RDONLY, 0);
10     Read(fd1, &c, 1);
11     Read(fd2, &c, 1);
12     printf("c = %c\n", c);
13     exit(0);
14 }
```

 **练习题 10.3** 就像前面那样，假设磁盘文件 `foobar.txt` 由 6 个 ASCII 码字符“foobar”组成。那么下列程序的输出是什么？

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd;
6      char c;
7
8      fd = Open("foobar.txt", O_RDONLY, 0);
9      if (Fork() == 0) {
10         Read(fd, &c, 1);
11         exit(0);
12     }
13     Wait(NULL);
14     Read(fd, &c, 1);
15     printf("c = %c\n", c);
16     exit(0);
17 }
```

10.9 I/O 重定向

Linux shell 提供了 I/O 重定向操作符，允许用户将磁盘文件和标准输入输出联系起来。例如，键入

```
linux> ls > foo.txt
```

使得 shell 加载和执行 ls 程序，将标准输出重定向到磁盘文件 foo.txt。就如我们将会在 11.5 节中看到的那样，当一个 Web 服务器代表客户端运行 CGI 程序时，它就执行一种相似类型的重定向。那么 I/O 重定向是如何工作的呢？一种方式是使用 dup2 函数。

```
#include <unistd.h>

int dup2(int oldfd, int newfd);
```

返回：若成功则为非负的描述符，若出错则为 -1。

dup2 函数复制描述符表表项 oldfd 到描述符表表项 newfd，覆盖描述符表表项 newfd 以前的内容。如果 newfd 已经打开了，dup2 会在复制 oldfd 之前关闭 newfd。

假设在调用 dup2(4,1)之前，我们的状态如图 10-12 所示，其中描述符 1(标准输出)对应于文件 A(比如一个终端)，描述符 4 对应于文件 B(比如一个磁盘文件)。A 和 B 的引用计数都等于 1。图 10-15 显示了调用 dup2(4,1)之后的情况。两个描述符现在都指向文件 B；文件 A 已经被关闭了，并且它的文件表和 v-node 表表项也已经被删除了；文件 B 的引用计数已经增加了。从此以后，任何写到标准输出的数据都被重定向到文件 B。

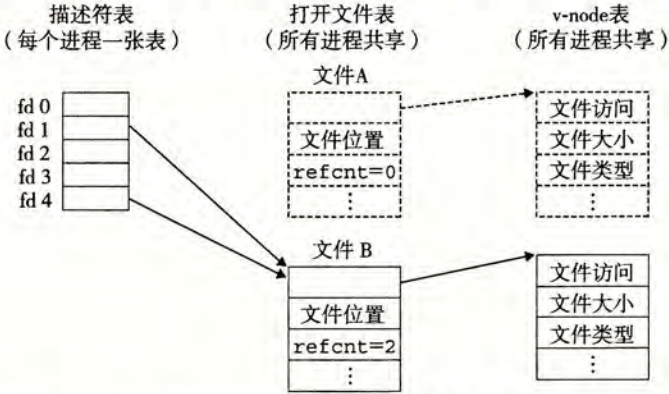


图 10-15 通过调用 dup2(4,1)重定向标准输出之后的内核数据结构。初始状态如图 10-12 所示

旁注 左边和右边的 hoinkies

为了避免和其他括号类型操作符比如 “]” 和 “[” 相混淆，我们总是将 shell 的 “>” 操作符称为 “右 hoinky”，而将 “<” 操作符称为 “左 hoinky”。

- 练习题 10.4 如何用 dup2 将标准输入重定向到描述符 5?
- 练习题 10.5 假设磁盘文件 foobar.txt 由 6 个 ASCII 码字符 “foobar” 组成，那么下列程序的输出是什么?

```
1 #include "csapp.h"
2
3 int main()
```